Replacing Multi-Step Assembly of Data Preparation Pipelines with One-Step LLM Pipeline Generation for Table QA

Fengyu Li
Zhejiang University
Hangzhou, China
fengyuli@zju.edu.cn

Junhao Zhu
Zhejiang University
Hangzhou, China
zhujunhao@zju.edu.cn

Kaishi Song
Zhejiang University
Hangzhou, China
sks@zju.edu.cn

Lu Chen
Zhejiang University
Hangzhou, China
luchen@zju.edu.cn

Zhongming Yao
Northeastern University
Shenyang, China
yaozming@stumail.neu.edu.cn

Tianyi Li
Aalborg University
Aalborg, Denmark
tianyi@cs.aau.dk

Christian S. Jensen
Aalborg University
Aalborg, Denmark
csj@cs.aau.dk

arXiv:2602.22721v2 [cs.DB] 1 Apr 2026

ABSTRACT

Table Question Answering (TQA) aims to answer natural language questions over structured tables. Large Language Models (LLMs) enable promising solutions to this problem, with operator-centric solutions that generate table manipulation pipelines in a multi-step manner offering state-of-the-art performance. However, these solutions rely on multiple LLM calls, resulting in prohibitive latencies and computational costs.

We propose Operation-R1, the first framework that trains lightweight LLMs (e.g., Qwen-4B/1.7B) via a novel variant of reinforcement learning with verifiable rewards to produce high-quality data-preparation pipelines for TQA in a single inference step. To train such an LLM, we first introduce a self-supervised rewarding mechanism to automatically obtain fine-grained pipeline-wise supervision signals for LLM training. We also propose variance-aware group resampling to mitigate training instability. To further enhance robustness of pipeline generation, we develop two complementary mechanisms: operation merge, which filters spurious operations through multi-candidate consensus, and adaptive rollback, which offers runtime protection against information loss in data transformation. Experiments on two benchmark datasets show that, with the same LLM backbone, Operation-R1 achieves average absolute accuracy gains of 8.83 and 4.44 percentage points over multi-step preparation baselines, with 79% table compression and a 2.2× reduction in monetary cost.

PVLDB Reference Format:
Fengyu Li, Junhao Zhu, Kaishi Song, Lu Chen, Zhongming Yao, Tianyi Li, and Christian S. Jensen. Replacing Multi-Step Assembly of Data Preparation Pipelines with One-Step LLM Pipeline Generation for Table QA. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/ZJU-DAILY/Operation-R1.git.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Table Question Answering (TQA) aims to answer natural language (NL) queries over (semi-)structured tables, spanning spreadsheets, databases, web tables. It empowers non-technical users to explore and extract insights without writing formal query languages such as SQL, enhances analytical productivity, and enables applications in finance [42], healthcare [1], and scientific research [23]. For example, TQA for business intelligence enables users to query sales, customer, and financial tables using natural language (e.g., "Which product had the highest growth in Q3?"), lowering the technical barrier to data analytics.

Given their semantic understanding and instruction-following capabilities Large Language Models (LLMs) have advanced TQA substantially. As illustrated in Fig.1, one line of LLM-based TQA methods employ *direct text-based reasoning* [34, 38]. These methods serialize tables into plain text and answer NL queries in a single step by reasoning directly over the text. However, they scale poorly to large tables because LLMs struggle to reliably identify relevant information within long contexts. Studies show that text-based reasoning deteriorates sharply as table size grows [8].

Another line of methods rely on *Text-to-SQL/code generation* [7, 24, 38, 40]. These methods translate NL queries into compositional executable programs (e.g., SQL or Python) and derive answers through program execution. Although often effective, they are fragile: LLMs easily introduce subtle errors into long, compound programs, causing executions to fail or produce corrupted results.

Motivated by limitations of above reasoning approaches, recent methods adopt *operator-centric* approaches [8, 32], where an LLM assembles a sequence of table manipulation operators (e.g., select, sort_by) to restructure input tables into more compact, question-relevant tables that facilitate the TQA the most. To discover effective table manipulation operations, these methods employ multi-step reasoning, where table manipulation operations are generated step-by-step in the context of input questions, tables, and intermediate orchestration results. This operator-centric approach facilitates more transparent, interpretable reasoning and enables methods that achieve state-of-the-art performance.

Despite their accuracy and interpretability advantages, operator-centric pipelines incur substantially higher latency and computational cost due to repeated LLM invocations. This overhead, which